TP: Módulo Materias

Programación IV · Tecnicatura Universitaria en Programación · UTN Reconquista
Valentino Serafini · Prof. Irina Agretti · Noviembre de 2026

1. Cómo pensé la entidad

Armé Materias copiando la forma de trabajar de Alumnos y Docentes, así que la estructura es la misma. La diferencia es que acá la clave es `codigo`, un texto como PRG4 o BD1, y todo se busca por ese código, nunca por el `_id`.

CAMPO	TIPO	OBLIG.	REGLAS
`codigo`	String	Sí	Único, sin espacios en los extremos, se guarda en mayúsculas, con al menos una letra, hasta 15 caracteres
`nombre`	String	Sí	Entre 3 y 100 caracteres
`descripcion`	String	No	No puede ser solo espacios, hasta 300 caracteres
`cargaHoraria`	Number	Sí	Entero mayor que cero
`estado`	String	Sí	activa o inactiva

Decidí guardar el código en mayúsculas porque si no, "prg4" y "PRG4" podían quedar como dos materias distintas, y eso no tiene sentido.

2. Backend

models/Materia.js

Es el esquema de Mongoose. A cada regla le puse su mensaje en español, así si algo llega a fallar en la base el usuario entiende qué pasó. El `unique: true` es lo que realmente impide que haya dos códigos iguales, aunque lleguen dos pedidos al mismo tiempo. En la carga horaria agregué `Number.isInteger` porque con `min` solo se colaba un 12.5.

backend/models/Materia.js

```
const mongoose = require("mongoose")

const materiaSchema = new mongoose.Schema({
    codigo: {
        type: String,
        required: [true, "El código es obligatorio"],
        unique: true,
        trim: true,
        uppercase: true,
        maxlength: [15, "El código no puede superar los 15 caracteres"],
        validate: {
            validator: valor => /[a-z]/i.test(valor),
            message: "El código debe contener al menos una letra"
        }
    },
    nombre: {
        type: String,
        required: [true, "El nombre es obligatorio"],
        trim: true,
        minlength: [3, "El nombre debe tener al menos 3 caracteres"],
        maxlength: [100, "El nombre no puede superar los 100 caracteres"]
    },
    descripcion: {
        type: String,
        trim: true,
        maxlength: [300, "La descripción no puede superar los 300 caracteres"],
        default: ""
```

```
    },
    cargaHoraria: {
        type: Number,
        required: [true, "La carga horaria es obligatoria"],
        min: [1, "La carga horaria debe ser mayor que cero"],
        validate: {
            validator: Number.isInteger,
            message: "La carga horaria debe ser un número entero"
        }
    },
    estado: {
        type: String,
        required: [true, "El estado es obligatorio"],
        enum: {
            values: ["activa", "inactiva"],
            message: "El estado solo puede ser activa o inactiva"
        }
    }
},
{
    versionKey: false
}
)

const Materia = mongoose.model("Materia", materiaSchema)

module.exports = Materia
```

controllers/materias.controller.js

Acá están las cinco operaciones. Antes de tocar la base, `validarMateria` revisa los datos y, si algo está mal, devuelve un 400 con el motivo. Los errores los junté en `manejarError` para no repetir código: un código duplicado devuelve 409, un error de validación 400 y cualquier otra cosa un 500 con un mensaje general, sin mostrar nada interno. Al modificar, solo controlo el duplicado si el código cambió, porque si la materia conserva el suyo no es un duplicado.

backend/controllers/materias.controller.js

```
const Materia = require("../models/Materia")

const ESTADOS = ["activa", "inactiva"]

function normalizarCodigo(codigo) {
    return String(codigo).trim().toUpperCase()
}

function validarMateria({ codigo, nombre, descripcion, cargaHoraria, estado }) {
    if (codigo === undefined || nombre === undefined || cargaHoraria === undefined || estado === undefined) {
        return "Código, nombre, carga horaria y estado son obligatorios"
    }
    if (typeof codigo !== "string") {
        return "El código debe ser un texto"
    }
    if (codigo.trim() === "") {
        return "El código no puede estar vacío"
    }
    if (!/[a-z]/i.test(codigo)) {
        return "El código debe contener al menos una letra"
    }
    if (typeof nombre !== "string" || nombre.trim() === "") {
        return "El nombre no puede estar vacío"
    }
    if (nombre.trim().length < 3 || nombre.trim().length > 100) {
        return "El nombre debe tener entre 3 y 100 caracteres"
    }
    if (descripcion !== undefined) {
        if (typeof descripcion !== "string") {
            return "La descripción debe ser un texto"
        }
        if (descripcion.length > 0 && descripcion.trim() === "") {
```

```
            return "La descripción no puede contener únicamente espacios"
        }
    }
    if (typeof cargaHoraria !== "number" || !Number.isInteger(cargaHoraria) || cargaHoraria <= 0) {
        return "La carga horaria debe ser un número entero mayor que cero"
    }
    if (!ESTADOS.includes(estado)) {
        return "El estado solo puede ser activa o inactiva"
    }
    return null
}

function manejarError(res, error) {
    if (error.code === 11000) {
        return res.status(409).json({
            mensaje: "Ya existe una materia con ese código"
        })
    }
    if (error.name === "ValidationError") {
        const primerError = Object.values(error.errors)[0]
        return res.status(400).json({
            mensaje: primerError.message
        })
    }
    console.error(error)
    res.status(500).json({
        mensaje: "No se pudo completar la operación en la base de datos"
    })
}

async function obtenerMaterias(req, res) {
    try {
        const materias = await Materia.find().sort({ codigo: 1 })
        res.json(materias)
    } catch (error) {
        manejarError(res, error)
    }
}

async function obtenerMateria(req, res) {
    try {
        const materia = await Materia.findOne({
            codigo: normalizarCodigo(req.params.codigo)
        })
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        res.json(materia)
    } catch (error) {
        manejarError(res, error)
    }
}

async function crearMateria(req, res) {
    const mensajeError = validarMateria(req.body)
    if (mensajeError) {
        return res.status(400).json({
            mensaje: mensajeError
        })
    }
    const { codigo, nombre, descripcion, cargaHoraria, estado } = req.body
    try {
        const existe = await Materia.findOne({
            codigo: normalizarCodigo(codigo)
        })
        if (existe) {
            return res.status(409).json({
                mensaje: "Ya existe una materia con ese código"
            })
```

```
        }
        const nuevaMateria = await Materia.create({
            codigo,
            nombre,
            descripcion,
            cargaHoraria,
            estado
        })
        res.status(201).json(nuevaMateria)
    } catch (error) {
        manejarError(res, error)
    }
}

async function actualizarMateria(req, res) {
    const mensajeError = validarMateria(req.body)
    if (mensajeError) {
        return res.status(400).json({
            mensaje: mensajeError
        })
    }
    const { codigo, nombre, descripcion, cargaHoraria, estado } = req.body
    try {
        const materia = await Materia.findOne({
            codigo: normalizarCodigo(req.params.codigo)
        })
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        const nuevoCodigo = normalizarCodigo(codigo)
        if (nuevoCodigo !== materia.codigo) {
            const existe = await Materia.findOne({ codigo: nuevoCodigo })
            if (existe) {
                return res.status(409).json({
                    mensaje: "Ya existe otra materia con ese código"
                })
            }
        }
        materia.set({
            codigo,
            nombre,
            descripcion: descripcion ?? "",
            cargaHoraria,
            estado
        })
        await materia.save()
        res.json(materia)
    } catch (error) {
        manejarError(res, error)
    }
}

async function eliminarMateria(req, res) {
    try {
        const materia = await Materia.findOneAndDelete(
            { codigo: normalizarCodigo(req.params.codigo) }
        )
        if (!materia) {
            return res.status(404).json({
                mensaje: "Materia no encontrada"
            })
        }
        res.json({ mensaje: "Materia eliminada correctamente" })
    } catch (error) {
        manejarError(res, error)
    }
}

module.exports = {
```

```
    obtenerMaterias,
    obtenerMateria,
    crearMateria,
    actualizarMateria,
    eliminarMateria
}
```

routes/materias.routes.js

Une cada método HTTP con su función. Al parámetro lo llamé `:codigo` para que quede claro que no es el `_id`.

backend/routes/materias.routes.js

```
const express = require("express")
const {
    obtenerMaterias,
    obtenerMateria,
    crearMateria,
    actualizarMateria,
    eliminarMateria } = require("../controllers/materias.controller")
const router = express.Router()

router.get("/", obtenerMaterias)

router.get("/:codigo", obtenerMateria)

router.post("/", crearMateria)

router.put("/:codigo", actualizarMateria)

router.delete("/:codigo", eliminarMateria)

module.exports = router
```

server.js

Solo agregué estas dos líneas: importar las rutas y registrarlas en `/materias`.

backend/server.js *líneas agregadas*

```
const materiasRoutes = require("./routes/materias.routes")
app.use("/materias", materiasRoutes)
```

3. Frontend

pages/Materias.jsx

Junta el formulario y la lista, igual que en Alumnos y Docentes. `actualizar` es un contador: cuando sube, la lista vuelve a pedir los datos. `materiaEditar` guarda la materia que estoy editando. Además agregué un aviso arriba para los mensajes de éxito y error. Si elimino una materia mientras edito otra, la edición no se pierde.

frontend-react/src/pages/Materias.jsx

```
import { ListaMaterias } from "../components/ListaMaterias"
import { useState } from "react"
import { Alert } from "@mui/material"
import FormularioMateria from "../components/FormularioMateria"

export default function Materias() {
const [actualizar, setActualizar] = useState(0)
const [materiaEditar, setMateriaEditar] = useState(null)
const [aviso, setAviso] = useState(null)

function operacionExitosa(mensaje) {
  setActualizar(valor => valor + 1)
```

```
  setAviso({ tipo: "success", mensaje })
}

function materiaGuardada(mensaje) {
  setMateriaEditar(null)
  operacionExitosa(mensaje)
}

function materiaEliminada(codigo, mensaje) {
  setMateriaEditar(actual => actual?.codigo === codigo ? null : actual)
  operacionExitosa(mensaje)
}

function mostrarError(mensaje) {
  setAviso({ tipo: "error", mensaje })
}

function seleccionarMateria(materia) {
  setMateriaEditar(materia)
  setAviso(null)
}

return (
   <>
   {aviso && (
    <Alert severity={aviso.tipo} onClose={() => setAviso(null)} sx={{ mb: 3 }}>
      {aviso.mensaje}
    </Alert>
   )}
   <FormularioMateria key={materiaEditar?.codigo ?? "nuevo"}
   onMateriaGuardada={materiaGuardada}
   materiaEditar={materiaEditar}
   onCancelar={() => setMateriaEditar(null)}
   />
   <ListaMaterias
   actualizar={actualizar}
   onMateriaEliminada={materiaEliminada}
   onEditarMateria={seleccionarMateria}
   onError={mostrarError}
   />
  </>
  )
}
```

components/FormularioMateria.jsx

Lo hice siguiendo `FormularioD`: el mismo formulario sirve para dar de alta y para editar. Antes de enviar, `validar` revisa los campos y marca el error debajo de cada uno. Si el servidor rechaza algo, muestro su mensaje, y si no hay conexión aviso que no se pudo conectar. Al editar, el `PUT` va al código original de la materia.

frontend-react/src/components/FormularioMateria.jsx

```
import { useState } from "react";
import { Alert, Box, Button, MenuItem, Paper, TextField, Typography } from "@mui/material"
const API_MATERIAS = "http://localhost:3000/materias"

const FORMULARIO_VACIO = {
    codigo: "",
    nombre: "",
    descripcion: "",
    cargaHoraria: "",
    estado: "activa"
}

function validar(formulario) {
    const errores = {}
    if (formulario.codigo.trim() === "") {
        errores.codigo = "El código es obligatorio"
    } else if (!/[a-z]/i.test(formulario.codigo)) {
```

```
        errores.codigo = "El código debe contener al menos una letra"
    }
    if (formulario.nombre.trim() === "") {
        errores.nombre = "El nombre es obligatorio"
    } else if (formulario.nombre.trim().length < 3) {
        errores.nombre = "El nombre debe tener al menos 3 caracteres"
    }
    if (formulario.descripcion.length > 0 && formulario.descripcion.trim() === "") {
        errores.descripcion = "La descripción no puede contener solo espacios"
    }
    if (formulario.cargaHoraria.trim() === "") {
        errores.cargaHoraria = "La carga horaria es obligatoria"
    } else if (!/^\d+$/.test(formulario.cargaHoraria.trim()) || Number(formulario.cargaHoraria) <= 0) {
        errores.cargaHoraria = "Debe ser un número entero mayor que cero"
    }
    if (!["activa", "inactiva"].includes(formulario.estado)) {
        errores.estado = "Seleccioná un estado válido"
    }
    return errores
}

function FormularioMateria({ onMateriaGuardada, materiaEditar, onCancelar }){
    const [formulario, setFormulario] = useState({
        codigo: materiaEditar?.codigo ?? "",
        nombre: materiaEditar?.nombre ?? "",
        descripcion: materiaEditar?.descripcion ?? "",
        cargaHoraria: String(materiaEditar?.cargaHoraria ?? ""),
        estado: materiaEditar?.estado ?? "activa"
    })
    const [errores, setErrores] = useState({})
    const [error, setError] = useState("")
    const [guardando, setGuardando] = useState(false)

    function manejarCambio(e) {
        const { name, value } = e.target

        setFormulario({
            ...formulario,
            [name]: value
        })
        setErrores({
            ...errores,
            [name]: ""
        })
    }

    async function guardar(e) {
        e.preventDefault()
        setError("")

        const erroresValidacion = validar(formulario)
        setErrores(erroresValidacion)
        if (Object.keys(erroresValidacion).length > 0) {
            return
        }

        setGuardando(true)

        const materia = {
            codigo: formulario.codigo.trim().toUpperCase(),
            nombre: formulario.nombre.trim(),
            descripcion: formulario.descripcion.trim(),
            cargaHoraria: Number(formulario.cargaHoraria),
            estado: formulario.estado
        }
        const editando = Boolean(materiaEditar)
        const url = editando
                    ? `${API_MATERIAS}/${encodeURIComponent(materiaEditar.codigo)}`
                    : API_MATERIAS
        try {
            const respuesta = await fetch(url, {
```

```
                method: editando ? "PUT" : "POST",
                headers: {"Content-Type":"application/json"},
                body: JSON.stringify(materia)
            })
            const datos = await respuesta.json().catch(() => null)

            if (!respuesta.ok) {
                throw new Error(datos?.mensaje || `No se pudo guardar la materia. Estado HTTP:
${respuesta.status}`)
            }

            setFormulario(FORMULARIO_VACIO)
            onMateriaGuardada(editando
                ? `Materia ${materia.codigo} modificada correctamente`
                : `Materia ${materia.codigo} registrada correctamente`)
        } catch (error) {
            console.error("Error al guardar:", error)
            setError(error instanceof TypeError
                ? "No se pudo conectar con el servidor"
                : error.message)
        } finally {
            setGuardando(false)
        }
    }
    function cancelar(){
        onCancelar()
    }
    return(
        <Paper
            component="section"
            elevation={0}
            sx={{
                p: { xs: 2.5, sm: 3.5 },
                mb: 3,
                border: 1,
                borderColor: "divider",
                borderRadius: 4,
                bgcolor: "background.paper"
            }}
        >
            <Typography component="h2" variant="h5" sx={{ mb: 2.5, fontWeight: 700 }}>
                {materiaEditar ? "Editar materia" : "Nueva Materia"}
            </Typography>
            <Box
                component="form"
                onSubmit={guardar}
                noValidate
                sx={{
                    display: "grid",
                    gridTemplateColumns: { xs: "1fr", sm: "repeat(2, minmax(0, 1fr))" },
                    gap: 2
                }}
            >
                <TextField
                    id="codigo"
                    name="codigo"
                    label="Código"
                    value={formulario.codigo}
                    onChange={manejarCambio}
                    error={Boolean(errores.codigo)}
                    helperText={errores.codigo}
                    slotProps={{ htmlInput: { maxLength: 15 } }}
                    required
                    fullWidth
                />
                <TextField
                    id="nombre"
                    name="nombre"
                    label="Nombre"
                    value={formulario.nombre}
                    onChange={manejarCambio}
```

```
                    error={Boolean(errores.nombre)}
                    helperText={errores.nombre}
                    slotProps={{ htmlInput: { maxLength: 100 } }}
                    required
                    fullWidth
                />
                <TextField
                    id="cargaHoraria"
                    name="cargaHoraria"
                    label="Carga horaria"
                    type="number"
                    value={formulario.cargaHoraria}
                    onChange={manejarCambio}
                    error={Boolean(errores.cargaHoraria)}
                    helperText={errores.cargaHoraria}
                    slotProps={{ htmlInput: { min: 1, step: 1 } }}
                    required
                    fullWidth
                />
                <TextField
                    id="estado"
                    name="estado"
                    label="Estado"
                    select
                    value={formulario.estado}
                    onChange={manejarCambio}
                    error={Boolean(errores.estado)}
                    helperText={errores.estado}
                    required
                    fullWidth
                >
                    <MenuItem value="activa">Activa</MenuItem>
                    <MenuItem value="inactiva">Inactiva</MenuItem>
                </TextField>
                <TextField
                    id="descripcion"
                    name="descripcion"
                    label="Descripción"
                    value={formulario.descripcion}
                    onChange={manejarCambio}
                    error={Boolean(errores.descripcion)}
                    helperText={errores.descripcion}
                    slotProps={{ htmlInput: { maxLength: 300 } }}
                    multiline
                    minRows={2}
                    fullWidth
                    sx={{ gridColumn: "1 / -1" }}
                />
                <Box
                    sx={{
                        gridColumn: "1 / -1",
                        display: "flex",
                        justifyContent: "flex-end",
                        flexWrap: "wrap",
                        gap: 1.5,
                        mt: 1
                    }}
                >
                    <Button type="submit" variant="contained" disabled={guardando}>
                        {guardando
                            ? "Guardando..."
                            : materiaEditar
                                ? "Guardar cambios"
                                : "Guardar"}
                    </Button>
                    {materiaEditar && (
                        <Button
                            type="button"
                            onClick={cancelar}
                            disabled={guardando}
                            variant="outlined"
```

```
                        >
                            Cancelar
                        </Button>
                    )}
                </Box>
                {error && (
                    <Alert severity="error" role="alert" sx={{ gridColumn: "1 / -1" }}>
                        {error}
                    </Alert>
                )}
            </Box>
        </Paper>
    )
}
export default FormularioMateria
```

components/ListaMaterias.jsx

Trae las materias de la API cada vez que cambia `actualizar`. Muestra un mensaje mientras carga, si falla la conexión o si no hay materias. En cada fila aparecen el código, el nombre, las horas, la descripción y el estado, con los botones para editar y eliminar. Antes de borrar pide confirmación.

frontend-react/src/components/ListaMaterias.jsx

```
import { useEffect, useState } from "react"
import { Alert, Box, Button, Chip, Paper, Stack, Typography } from "@mui/material"

const API_MATERIAS = "http://localhost:3000/materias"

export function ListaMaterias({ actualizar, onMateriaEliminada, onEditarMateria, onError }) {
    const [materias, setMaterias] = useState([])
    const [cargando, setCargando] = useState(true)
    const [error, setError] = useState("")

    async function eliminarMateria(materia){
        const confirmar = window.confirm(`¿Seguro que querés eliminar la materia ${materia.codigo} - ${materia.nombre}?`)
        if (!confirmar) {
            return
        }
        try {
            const respuesta = await fetch(`${API_MATERIAS}/${encodeURIComponent(materia.codigo)}`,
                { method: "DELETE" }
            )
            const datos = await respuesta.json().catch(() => null)
            if (!respuesta.ok) {
                throw new Error(datos?.mensaje || `No se pudo eliminar. Estado HTTP: ${respuesta.status}`)
            }
            onMateriaEliminada(materia.codigo, `Materia ${materia.codigo} eliminada correctamente`)
        } catch (error) {
            console.error("Error al eliminar: ", error)
            onError(error instanceof TypeError
                ? "No se pudo conectar con el servidor"
                : error.message)
        }
    }
    useEffect(() => {
        async function cargarMaterias(){
            setCargando(true)
            setError("")

            try {
                const respuesta = await fetch(API_MATERIAS)

                if (!respuesta.ok){
                    throw new Error(`No se pudo obtener la lista. Estado HTTP: ${respuesta.status}`)
                }

                const datos = await respuesta.json()
```

```
                setMaterias(datos)
            } catch(error) {
                console.error("Error de conexión", error)
                setError("No se pudo cargar la lista de materias. Verificá que el servidor esté funcionando.")
            } finally {
                setCargando(false)
            }
        }
        cargarMaterias()
    }, [actualizar])

    if (cargando) {
        return (
            <Paper component="section" elevation={0} sx={{ p: 3, border: 1, borderColor: "divider", borderRadius: 4 }}>
                <Typography color="text.secondary">Cargando materias...</Typography>
            </Paper>
        )
    }
    if (error) {
        return <Alert severity="error" role="alert">{error}</Alert>
    }
    return (
        <Paper
            component="section"
            elevation={0}
            sx={{
                p: { xs: 2.5, sm: 3.5 },
                border: 1,
                borderColor: "divider",
                borderRadius: 4,
                bgcolor: "background.paper"
            }}
        >
            <Typography component="h2" variant="h5" sx={{ mb: 2.5, fontWeight: 700 }}>
                Materias
            </Typography>
            {materias.length === 0
                ? (
                    <Typography color="text.secondary">
                        No hay materias registradas.
                    </Typography>
                )
                : (
                    <Stack
                        component="ul"
                        spacing={1.5}
                        sx={{ listStyle: "none", p: 0, m: 0 }}
                    >
                        {materias.map(materia => (
                            <Paper
                                component="li"
                                key={materia.codigo}
                                elevation={0}
                                sx={{
                                    p: { xs: 2, sm: 2.5 },
                                    border: 1,
                                    borderColor: "divider",
                                    borderRadius: 3,
                                    display: "flex",
                                    alignItems: { xs: "stretch", sm: "center" },
                                    justifyContent: "space-between",
                                    flexDirection: { xs: "column", sm: "row" },
                                    gap: 2
                                }}
                            >
                                <Stack
                                    direction="row"
                                    spacing={1.5}
                                    sx={{ alignItems: "center", flexWrap: "wrap" }}
                                >
```

```
                                <Chip label={materia.codigo} color="primary" variant="outlined" size="small"
/>
                                <Box>
                                    <Typography fontWeight={600}>{materia.nombre}</Typography>
                                    <Typography variant="body2" color="text.secondary">
                                        {materia.cargaHoraria} horas
                                        {materia.descripcion && ` · ${materia.descripcion}`}
                                    </Typography>
                                </Box>
                                <Chip
                                    label={materia.estado === "activa" ? "Activa" : "Inactiva"}
                                    color={materia.estado === "activa" ? "success" : "default"}
                                    size="small"
                                />
                            </Stack>
                            <Stack direction="row" spacing={1} sx={{ justifyContent: "flex-end" }}>
                                <Button
                                    type="button"
                                    onClick={() => onEditarMateria(materia)}
                                    variant="outlined"
                                    size="small"
                                >
                                    Editar
                                </Button>
                                <Button
                                    type="button"
                                    onClick={() => eliminarMateria(materia)}
                                    variant="outlined"
                                    color="error"
                                    size="small"
                                >
                                    Eliminar
                                </Button>
                            </Stack>
                        </Paper>
                    ))}
                </Stack>
            )}
        </Paper>
    )
}
```

App.jsx y Header.jsx

Sumé la ruta dentro de `LayoutPpal`, así la página tiene el mismo encabezado, pie y selector de tema que las demás, y agregué Materias al menú.

frontend-react/src/App.jsx — *líneas agregadas*

```
 7 import Materias from "./pages/Materias"
19 <Route path="materias" element={<Materias />} />
```

frontend-react/src/components/Header.jsx — *fragmento agregado*

```
<Link
    component={RouterLink}
    to="/materias"
    color="text.primary"
    underline="none"
    sx={{ px: { xs: 1, sm: 1.5 }, py: 1, borderRadius: 2, "&:hover": { bgcolor: "action.hover" } }}
>
    Materias
</Link>
```

4. Pruebas

API con Rest Client (pruebas.http)

Antes de pasar a React probé el backend con estas peticiones, y después de cada cambio volví a listar para confirmar que se hubiera guardado.

#	PRUEBA	PETICIÓN	ESTADO
1	Listar con la colección vacía	`GET /materias`	`200`
2	Registrar varias y listarlas	`POST + GET /materias`	`201 / 200`
3	Consultar por código	`GET /materias/PRG4`	`200`
4	Consultar un código que no existe	`GET /materias/XYZ9`	`404`
5	Alta con datos válidos	`POST /materias`	`201`
6	Alta sin código, con espacios, con 123 o con "123"	`POST /materias`	`400`
7	Alta sin nombre	`POST /materias`	`400`
8	Alta sin carga horaria	`POST /materias`	`400`
9	Carga horaria 0, negativa o decimal	`POST /materias`	`400`
10	Código repetido	`POST /materias`	`409`
11	Estado que no existe	`POST /materias`	`400`
12	Cambiar nombre y descripción	`PUT /materias/BD1`	`200`
13	Cambiar carga horaria y estado	`PUT /materias/BD1`	`200`
14	Modificar una materia que no existe	`PUT /materias/XYZ9`	`404`
15	Ponerle el código de otra materia	`PUT /materias/BD1`	`409`
16	Eliminar y volver a consultar	`DELETE + GET /materias/PRG3`	`200 / 404`
17	Eliminar una que no existe	`DELETE /materias/PRG3`	`404`
18	Listar al final para ver los cambios	`GET /materias`	`200`

Concurrencia (pruebaConcurrencia.js)

Con Rest Client no se pueden mandar pedidos al mismo tiempo, así que hice este script. Manda cinco altas con el mismo código a la vez y siempre entra una sola: un 201 y cuatro 409. Al final borra la materia de prueba.

backend/pruebaConcurrencia.js

```
const API_MATERIAS = "http://localhost:3000/materias"
const CODIGO = "CONC1"
const CANTIDAD = 5

const materia = {
    codigo: CODIGO,
    nombre: "Prueba de concurrencia",
    cargaHoraria: 10,
    estado: "activa"
}

function registrar() {
    return fetch(API_MATERIAS, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(materia)
    })
```

```
}

async function probarConcurrencia() {
    try {
        const previa = await fetch(`${API_MATERIAS}/${CODIGO}`)
        if (previa.ok) {
            console.log(`La materia ${CODIGO} ya existe. Eliminala antes de correr la prueba.`)
            return
        }

        console.log(`Enviando ${CANTIDAD} POST simultáneos con el código ${CODIGO}...`)
        const respuestas = await Promise.all(
            Array.from({ length: CANTIDAD }, registrar)
        )

        for (const [indice, respuesta] of respuestas.entries()) {
            const datos = await respuesta.json()
            console.log(`Pedido ${indice + 1}: ${respuesta.status} - ${datos.mensaje ?? "materia registrada"}`)
        }

        const creadas = respuestas.filter(respuesta => respuesta.status === 201).length
        console.log(`Materias registradas: ${creadas} de ${CANTIDAD}`)

        const borrado = await fetch(`${API_MATERIAS}/${CODIGO}`, { method: "DELETE" })
        console.log(`Limpieza: DELETE ${CODIGO} - ${borrado.status}`)
    } catch (error) {
        console.error("No se pudo conectar con el servidor. Verificá que el backend esté funcionando.")
    }
}

probarConcurrencia()
```

Desde React

#	PRUEBA	QUÉ PASÓ	
1	Registrar una materia válida desde React	Sale el aviso de éxito y el formulario queda vacío	**OK**
2	Verificar que se guarde en MongoDB	Quedó guardada en la base SGA, colección materias	**OK**
3	Comprobar que aparezca en el listado	Aparece sin recargar la página	**OK**
4	Registrar un código repetido	"Ya existe una materia con ese código"	**OK**
5	Guardar datos inválidos	Cada campo marca su error y no se manda nada	**OK**
6	Modificar una materia	Al tocar Editar se cargan los datos y se guarda bien	**OK**
7	Ver que la lista refleje el cambio	La fila muestra los datos nuevos	**OK**
8	Eliminar una materia	Pide confirmación y la fila desaparece	**OK**
9	Operar sobre una materia que ya no existe	"Materia no encontrada"	**OK**
10	Sin conexión con el backend	"No se pudo cargar la lista de materias"	**OK**
11	Alumnos y Docentes siguen andando	Las dos listas cargan bien	**OK**